

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA1401

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 10%

QUESTIONS: 10

**Duration :60 Mins
In Class
Total Marks:50**

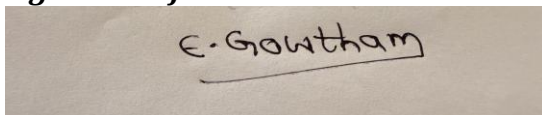
Annexure A

DATA INTERPRETATION

Code of Conduct:

*I Enamala Gowtham/192311190 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

A photograph of a piece of paper with the handwritten signature "E. Gowtham" in black ink. The signature is written in a cursive style and is underlined.

Annexure A

DATA INTERPRETATION

Q1. Given the three-address code:

```
t1 = a * b
t2 = t1 + c
t3 = t2 - d
```

- (i) Analyze the sequence of computations performed by the intermediate code.
- (ii) Construct the equivalent expression tree for the given code.
- (iii) Explain how temporary variables (t1, t2) help reduce recomputation and improve execution efficiency. (5 Marks)

Q2. Given the symbol table entries:

Variable	Type	Size
p	int	4
q	float	8
r	char	1
s	double	8

- (i) Interpret how memory is allocated for each variable.
- (ii) Calculate the total memory requirement for the variables.
- (iii) Explain the effect of memory alignment and padding on the overall storage size. (5 Marks)

Q3. Given the intermediate code:

```
t1 = x + y
t2 = t1 * z
t3 = x + y
t4 = t3 - w
```

- (i) Trace the computation step-by-step.
- (ii) Identify opportunities for common subexpression elimination.
- (iii) Rewrite the optimized intermediate code after removing redundant computations. (5 Marks)

Q4. Given the condition:

if (a < b || c > d)

Intermediate code:

```
if a < b goto L1
if c > d goto L1
goto L2
L1: ...
```

- (i) Interpret the control flow represented by the intermediate code.
- (ii) Explain how short-circuit evaluation is implemented.
- (iii) Illustrate the execution flow for different values of a, b, c, and d. (5 Marks)

Q5. Given:

Instruction	Code
120	if p > q goto _
121	goto _

Backpatch lists:

```
truelist = {120}
falselist = {121}
```

- (i) Analyze how backpatching is applied to the given instructions.
- (ii) Fill in the appropriate target labels for the pending jumps.
- (iii) Explain how control flow is resolved using the truelist and falselist. (5 Marks)

Q6. Given the expression:

result = a * b + c / d

- (i) Generate the corresponding three-address code.
- (ii) Explain the order of evaluation based on operator precedence.
- (iii) Describe the role of temporary variables in evaluating the expression. (5 Marks)

Q7. Given:

```
switch(choice)
{
    case 1: OPEN;
    case 2: SAVE;
```

```
    case 3: EXIT;  
}
```

(i) Translate the switch statement into intermediate code using conditional jumps.

(ii) Explain the execution of conditional branches for each case.

(iii) Describe how a jump table can optimize the execution of the switch statement.

(5 Marks)

Q8. Given the function call:

```
total = compute(x, y)
```

Intermediate representation:

```
param x  
param y  
call compute, 2
```

(i) Explain the parameter passing mechanism used in the procedure call.

(ii) Describe the structure and purpose of the activation record.

(iii) Explain how the return value is transferred back to the calling function. (5 Marks)

Q9. Given:

```
if (m > n && p < q)
```

Intermediate code:

```
if m > n goto L1  
goto L3
```

```
L1: if p < q goto L2  
goto L3
```

(i) Interpret the control flow represented by the code.

(ii) Explain how true and false lists are maintained during code generation.

(iii) Describe how backpatching resolves unresolved jump targets. (5 Marks)

Q10. Given the intermediate code:

```
t1 = p * q  
t2 = t1 + r  
t3 = t2 * s
```

- (i) Analyze the data dependencies among the statements.
- (ii) Explain how computed values are reused through temporary variables.
- (iii) Construct the corresponding data-flow representation and identify possible optimization opportunities. (5 Marks)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding ; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q1. Three-Address Code

Given:

$t1 = a * b$

$t2 = t1 + c$

$t3 = t2 - d$

(i) Analyze the sequence of computations

The given three-address code breaks the complete expression into smaller instructions. Each instruction contains at most one operator and uses temporary variables for intermediate results.

Step 1:

$t1 = a * b$

The values of a and b are multiplied. The result is stored in temporary variable $t1$.

Step 2:

$t2 = t1 + c$

The previously calculated value $t1$ is added to c . The result is stored in $t2$.

Step 3:

$t3 = t2 - d$

The value stored in $t2$ is subtracted from d according to the expression represented by the code. The result is stored in $t3$.

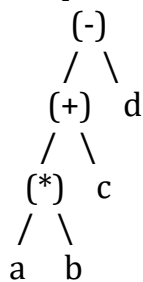
Thus, the complete expression represented is:

$t3 = (a * b) + c - d$

The evaluation order follows operator precedence: multiplication is performed before addition and subtraction.

(ii) Equivalent expression tree

The expression tree is:



The leaf nodes represent operands, while the internal nodes represent operators.

- $*$ represents multiplication.
- $+$ represents addition.
- $-$ represents subtraction.

The root node $-$ represents the final operation.

(iii) Role of temporary variables

Temporary variables are important in intermediate code generation.

- $t1$ stores the result of $a*b$.
- $t2$ stores the result of $t1+c$.
- $t3$ stores the final result.
- Intermediate values do not need to be recalculated.
- They make complex expressions easier for the compiler to process.

- They allow optimization techniques such as common subexpression elimination and dead-code elimination.
- During target-code generation, temporary values can potentially be stored in processor registers.

For example, if $a*b$ were needed again, the compiler could reuse $t1$ instead of calculating $a*b$ again.

Conclusion: Three-address code converts a complex expression into simple instructions, making the program easier to analyze, optimize, and translate into machine code.

Q2. Symbol Table and Memory Allocation

Given:

Variable	Type	Size
p	int	4 bytes
q	float	8 bytes
r	char	1 byte
s	double	8 bytes

(i) Interpretation of memory allocation

The symbol table contains information required by the compiler to manage variables.

For each variable, the compiler stores information such as:

- Variable name
- Data type
- Size
- Memory location/address
- Scope
- Storage information

Based on the given table:

$p \rightarrow \text{int} \rightarrow 4 \text{ bytes}$

$q \rightarrow \text{float} \rightarrow 8 \text{ bytes}$

$r \rightarrow \text{char} \rightarrow 1 \text{ byte}$

$s \rightarrow \text{double} \rightarrow 8 \text{ bytes}$

Therefore, the compiler must reserve sufficient storage for each variable.

(ii) Total memory requirement

The total data size is:

$p = 4 \text{ bytes}$

$q = 8 \text{ bytes}$

$r = 1 \text{ byte}$

$s = 8 \text{ bytes}$

Therefore:

Total = $4 + 8 + 1 + 8$

= 21 bytes

Hence, the basic memory requirement is **21 bytes**.

(iii) Memory alignment and padding

Modern processors often require data to be stored at particular memory boundaries. This is called **memory alignment**.

For example, an 8-byte object may preferably start at an address divisible by 8. Because of this, the compiler may insert unused bytes between variables. These unused bytes are called **padding**.

For example, conceptually:

p → 4 bytes

padding → possible

q → 8 bytes

r → 1 byte

padding → possible

s → 8 bytes

Therefore, the actual memory occupied may be greater than the simple total of 21 bytes.

The exact amount depends on:

- Processor architecture
- Compiler
- Alignment rules
- Data type requirements
- Structure layout rules

Conclusion: The variables require 21 bytes of actual data, but alignment and padding can increase the total storage allocated by the compiler.

Q3. Common Subexpression Elimination

Given:

t1 = x + y

t2 = t1 * z

t3 = x + y

t4 = t3 - w

(i) Trace the computation

The first instruction calculates:

t1 = x + y

The second instruction uses t1:

t2 = t1 * z

The third instruction again calculates:

t3 = x + y

The fourth instruction uses t3:

t4 = t3 - w

The important observation is that x+y is calculated twice.

(ii) Identify common subexpression

The common subexpression is:

x + y

It occurs in:

t1 = x + y

and:

$t3 = x + y$

If the values of x and y have not changed between these instructions, both calculations produce the same result.

Therefore, recalculating $x+y$ is unnecessary.

(iii) Optimized code

Original:

$t1 = x + y$

$t2 = t1 * z$

$t3 = x + y$

$t4 = t3 - w$

Optimized:

$t1 = x + y$

$t2 = t1 * z$

$t4 = t1 - w$

The compiler removes:

$t3 = x + y$

because $t1$ already contains the required result.

Advantages

Common subexpression elimination:

- Reduces the number of instructions.
- Avoids repeated calculations.
- Reduces CPU execution time.
- May reduce register and memory usage.
- Produces more efficient target code.

Conclusion: Reusing $t1$ instead of recalculating $x+y$ improves the efficiency of the intermediate code.

Q4. Short-Circuit Evaluation

Given:

if ($a < b$ || $c > d$)

Intermediate code:

if $a < b$ goto L1

if $c > d$ goto L1

goto L2

L1: ...

(i) Interpretation of control flow

The logical expression is:

$(a < b)$ OR $(c > d)$

The compiler first evaluates $a < b$.

If:

$a < b = \text{TRUE}$

the complete OR expression is already true, so control immediately goes to L1.

If:

$a < b = \text{FALSE}$

the compiler evaluates $c > d$.

If $c > d$ is true, control goes to L1.

If both conditions are false, control goes to L2.

(ii) Short-circuit implementation

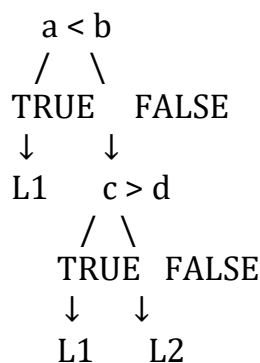
Short-circuit evaluation means that the compiler does not evaluate unnecessary conditions.

For OR:

TRUE OR anything = TRUE

Therefore, if the first condition is true, the second condition does not need to be checked.

The control flow is:



(iii) Different execution cases

Case 1:

$a = 10, b = 20$

Then:

$a < b \rightarrow \text{TRUE}$

Control immediately moves to L1.

Case 2:

Suppose:

$a = 20, b = 10$

$c = 30, d = 20$

Then:

$a < b \rightarrow \text{FALSE}$

$c > d \rightarrow \text{TRUE}$

Therefore, control moves to L1.

Case 3:

Suppose:

$a = 20, b = 10$

$c = 10, d = 20$

Both conditions are false, so control moves to L2.

Conclusion: Short-circuit evaluation reduces unnecessary computation and is especially useful when the second condition is expensive to evaluate.

Q5. Backpatching

Given:

120: if p > q goto _

121: goto _

Lists:

truelist = {120}

falselist = {121}

(i) Backpatching

During intermediate code generation, the compiler may not know the final destination of a jump.

Therefore, it initially produces:

if p > q goto _

goto _

The _ represents an unresolved target.

The compiler maintains two lists:

truelist = {120}

falselist = {121}

These lists store the instruction numbers whose targets need to be filled later.

(ii) Target labels

Suppose the compiler determines that:

L1 = true block

L2 = false block

After backpatching:

120: if p > q goto L1

121: goto L2

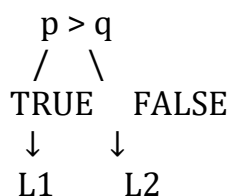
Thus:

truelist → 120 → L1

falselist → 121 → L2

(iii) Control-flow resolution

The control flow is:



If p > q is true, instruction 120 transfers control to L1.

If it is false, instruction 121 transfers control to L2.

Importance of backpatching

Backpatching is useful because:

- It supports one-pass intermediate code generation.
- It handles forward jumps.
- It simplifies Boolean expression translation.
- It helps generate code for if, while, for, and switch constructs.
- It connects incomplete jumps with their final destinations.

Conclusion: Backpatching resolves incomplete jump instructions after their destination labels become available.

Q6. Three-Address Code

Given:

$\text{result} = a * b + c / d$

(i) Three-address code

The expression is divided into smaller operations:

$t1 = a * b$

$t2 = c / d$

$t3 = t1 + t2$

$\text{result} = t3$

(ii) Operator precedence

Multiplication and division have higher precedence than addition.

Therefore:

$a * b$

is evaluated first, and:

c / d

is also evaluated before the addition.

Finally:

$(a * b) + (c / d)$

is calculated.

The order can therefore be represented as:

$a * b \rightarrow t1$

$c / d \rightarrow t2$

$t1 + t2 \rightarrow t3$

$t3 \rightarrow \text{result}$

(iii) Role of temporary variables

Temporary variables store intermediate results.

$t1 \rightarrow \text{result of } a * b$

$t2 \rightarrow \text{result of } c / d$

$t3 \rightarrow \text{result of } t1 + t2$

Without temporary variables, it would be difficult to represent the expression using simple three-address instructions.

Temporary variables also:

- Simplify code generation.
- Help optimization.
- Make data dependencies explicit.
- Can potentially be mapped to CPU registers.

Conclusion: Three-address code represents the expression according to operator precedence and divides it into simple operations suitable for optimization and target-code generation.

Q7. Switch Statement

Given:

```
switch(choice)
{
    case 1: OPEN;
    case 2: SAVE;
    case 3: EXIT;
}
```

(i) Intermediate code using conditional jumps

The switch can be translated as:

```
if choice == 1 goto L1
if choice == 2 goto L2
if choice == 3 goto L3
goto L4
```

```
L1:
    OPEN
    goto L4
```

```
L2:
    SAVE
    goto L4
```

```
L3:
    EXIT
    goto L4
```

L4:
Here L4 represents the end of the switch statement.

(ii) Execution of branches

```
If:
choice = 1
then:
choice == 1 → TRUE
    ↓
    L1
    ↓
    OPEN
```

```
If:
choice = 2
the first comparison fails and the second succeeds:
choice == 1 → FALSE
choice == 2 → TRUE
    ↓
    L2
```

param y

prepare the two arguments for the function.

Then:

call compute, 2

invokes the function compute with two parameters.

The actual implementation depends on the calling convention. Parameters may be passed through:

- Registers
- Stack
- Memory locations

The important purpose is to transfer the values of x and y from the calling function to compute.

(ii) Activation record

An activation record is a block of memory created for a function call. It is commonly associated with a stack frame.

A typical activation record contains:

```
+-----+
| Parameters      |
+-----+
| Return value    |
+-----+
| Return address  |
+-----+
| Saved registers |
+-----+
| Local variables |
+-----+
| Temporary values|
+-----+
```

It helps the compiler and runtime system manage the function's execution.

Each function invocation can have its own activation record.

(iii) Return value

After compute(x,y) finishes, it produces a result.

Conceptually:

compute(x,y)

|
↓

return value

|
↓

total

The return value is normally transferred using a register or another location defined by the target machine's calling convention.

Therefore:

total = returned value

Conclusion: Procedure-call intermediate code manages argument passing, function invocation, activation records, and return-value transfer.

Q9. Boolean AND and Backpatching

Given:

if (m > n && p < q)

Intermediate code:

if m > n goto L1

goto L3

L1:

if p < q goto L2

goto L3

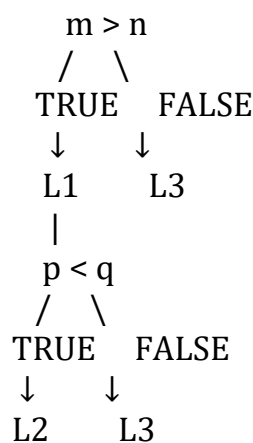
(i) Control flow

The expression is:

(m > n) AND (p < q)

AND requires both conditions to be true.

The control flow is:



Therefore:

- m > n false → condition false → L3.
- m > n true → evaluate p < q.
- p < q true → complete condition true → L2.
- p < q false → condition false → L3.

(ii) True and false lists

During Boolean expression translation, the compiler maintains lists of incomplete jumps.

For the first condition:

m > n

the true branch must continue to the second condition.

The false branch must go directly to the false destination.

For the second condition:

p < q

the true branch goes to L2, while the false branch goes to L3.

These pending jumps are stored in true and false lists until the final destinations are known.

(iii) Backpatching

Initially, the compiler may generate:

```
if m > n goto _  
goto _
```

Later, the destinations are known:

```
if m > n goto L1  
goto L3
```

Similarly:

```
if p < q goto L2  
goto L3
```

Backpatching therefore connects incomplete jump instructions with the correct labels.

Conclusion: Backpatching is essential for implementing Boolean expressions and structured control flow because it allows unresolved jumps to be completed after the target locations are determined.

Q10. Data Dependencies and Data Flow

Given:

```
t1 = p * q  
t2 = t1 + r  
t3 = t2 * s
```

(i) Data dependencies

The instructions have a clear dependency chain.

First:

```
t1 = p * q
```

The value of t1 is needed by the second instruction:

```
t2 = t1 + r
```

Then t2 is required by:

```
t3 = t2 * s
```

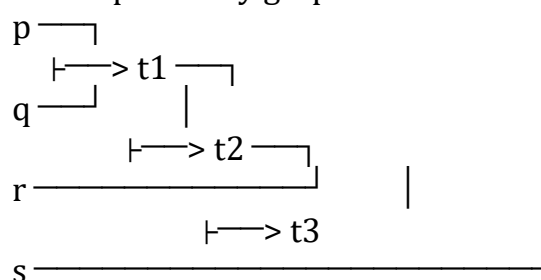
Therefore:

```
p, q → t1
```

```
t1, r → t2
```

```
t2, s → t3
```

The dependency graph is:



(ii) Reuse through temporary variables

The compiler does not need to recalculate intermediate results.

For example:

$t1 = p * q$

calculates $p*q$ once.

The result is reused in:

$t2 = t1 + r$

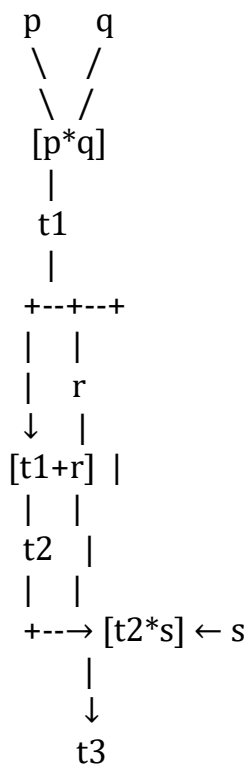
Similarly, $t2$ is reused by:

$t3 = t2 * s$

This makes the data dependencies explicit and simplifies later optimization.

(iii) Data-flow representation

A simple data-flow graph is:



A simplified dependency chain is:

$t1 = p * q$

↓

$t2 = t1 + r$

↓

$t3 = t2 * s$

Optimization opportunities

Possible optimizations include:

1. Constant folding

If operands are constants, the compiler can calculate them during compilation.

2. Common subexpression elimination

If an expression such as $p*q$ occurs elsewhere without changes to p or q , its result can be reused.

3. Register allocation

Frequently used temporary values such as t1 and t2 can potentially be stored in registers.

4. Dead-code elimination

If t3 is never used later, the compiler may eliminate the unnecessary computation.

5. Temporary optimization

During target-code generation, the compiler may reduce the number of temporary storage locations when it is safe to do so.

However, the dependency chain:

$t1 \rightarrow t2 \rightarrow t3$

must be respected because each operation depends on the previous result.

Conclusion: Data-flow analysis identifies how values move between instructions. It helps the compiler determine which computations can be reused, eliminated, or efficiently scheduled.